04

Account / Device Setup Manual

APK, identities, claiming and handover

EDITION 1.0 | 27 OCTOBER 2026 | BASELINE d623cce

READERS / Owners + administrators

Private credentials stay private.

Prepared for the October 28 prototype session. Physical acceptance is pending.

IN THIS GUIDE

Understand the three identities 02

Install the standalone Android app 03

Create the owner account and claim 04

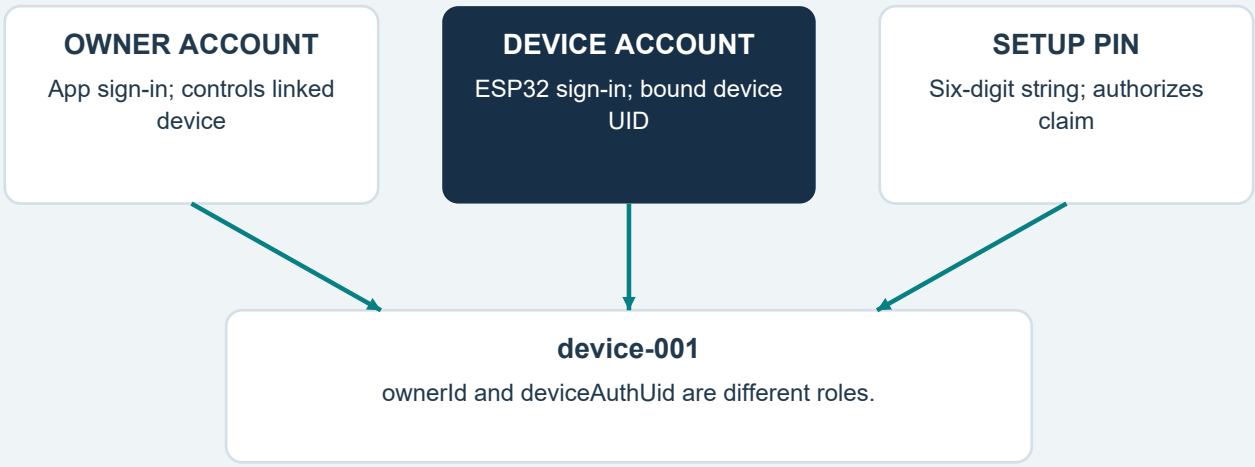
Administrator: verify or provision a device 05

Technician: connect the actual ESP32 06

Verify the phone, ownership and live data 07

Transfer from test owner to the client 08

Understand the three identities



Keep passwords and the private PIN separate from published documentation.

Item	Purpose	Who uses it
Owner account	Signs in to the Android app and owns the linked device through ownerId.	Operator / client.
Dedicated device account	Authenticates the ESP32; its Auth UID must match deviceAuthUid.	Firmware, managed by the administrator.
Six-digit setup PIN	Authorizes claiming a configured Device ID. It is not either account's password.	Intended owner during claim.

The current prototype Device ID is **device-001** in project **hyfepoul-dev**. The identifier must match the firmware and database exactly. QR-code provisioning is a possible future enhancement, not this setup flow.

What already exists in this edition

As recorded on 27 October 2026, the dedicated device account, matching deviceAuthUid, device record and private six-digit string PIN were provisioned. REST sign-in and a binding read passed. Do not recreate or overwrite these records merely because this manual includes setup steps.

The actual client account and physical-phone claim still need validation. The device can remain offline after a successful claim until the configured ESP32 connects and sends valid data. Wi-Fi placeholders must be replaced privately and the ESP32 rebuilt before the hardware connection test.

WHERE IS THE PIN?

The PIN is held privately by the authorized project administrator at deviceSecrets/device-001/setupPin. There is no public/default PIN in this manual. The owner should receive it privately. The administrator can securely replace it if it is lost; this is separate from an account password reset.

Source: docs/device-provisioning-guide.md; development-status.md; RTDB rules.

Install the standalone Android app

Use the prepared production build

The local file is **mobile-app/android/app/build/outputs/apk/release/app-release-production.apk**. It has an embedded JavaScript bundle, uses the production Firebase project and does not need Expo Go, Metro or a laptop server. This edition is a direct-install build signed with a development key, not a Play Store publication.

1. Obtain the APK from the authorized project maintainer through the agreed private transfer method. Do not install an unknown copy with a similar filename.
2. Have the maintainer verify its SHA-256 against the build record below. A rebuilt APK will need a new recorded checksum.
3. On the phone, open the file. If Android asks, allow installation from that specific trusted installer. Android wording varies by manufacturer.
4. Install and launch HyFePoul. Restore the installer permission afterward if it is no longer needed.
5. Confirm the app opens independently of Expo Go or Metro. Make sure the phone has internet for authentication and claiming.

Build fingerprint for this edition

```
SHA-256:
86F677D1B5B7D1FBD72CED83C75383F1
8961C0B1EDDEBC975757ADE0B11492BF
```

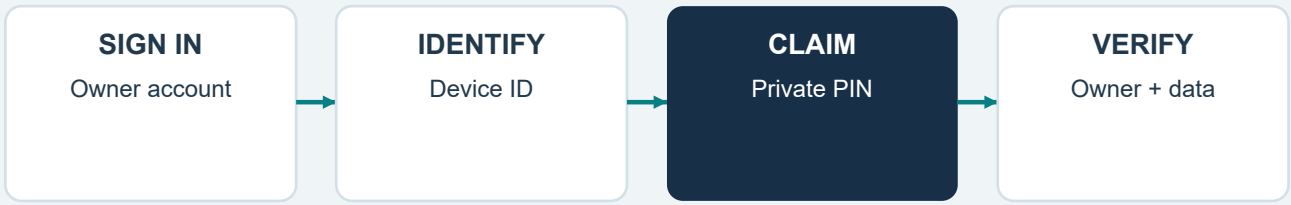
The hash is wrapped across two lines for printing; join them without spaces. Recorded size: 82,507,509 bytes. The source build record also verifies the signature and cleartext-disabled manifest. These are build checks, not a completed physical-phone acceptance test.

If installation is blocked

Record the exact Android message. A signature mismatch with an older installed build may require a coordinated uninstall/reinstall; uninstalling deletes local app data. Confirm access to the intended owner account before doing so. Do not disable broad phone protections or switch to a local-emulator build to bypass a production setup problem.

Source: docs/development-status.md; APK release/verification records referenced there.

Create the owner account and claim



Account ownership first. Hardware connectivity is a separate check.

Use the exact configured ID; never substitute an Auth UID for the Device ID.

Owner steps

1. Open HyFePoul. Use **Sign Up** to create the intended owner account with an email, password and confirmation, or sign in to its existing account.
2. Keep this account separate from the ESP32 device account. The temporary test-owner account is not automatically the final client account.
3. On **Claim Device**, type **device-001** exactly. Check the keyboard has not changed the case; a Device ID is not an email or an Auth UID.
4. Enter the privately supplied six-digit setup PIN, preserving any leading zero, and select **Claim Device** once.
5. Wait for the main app tabs. If the app reports an error, capture the message without the PIN and ask the administrator to inspect both ownership and the retained request before retrying.

WHAT SUCCESS LOOKS LIKE

The main app opens for the correct owner and Device ID. The administrator can confirm devices/device-001/ownerId equals the owner's Auth UID. Offline or Waiting for first device update can still be correct before the hardware connects.

The current app claim consists of two steps: it creates claimRequests/{id}, then establishes ownerId. A partial failure needs inspection. Do not repeatedly create accounts or ask the administrator to make the database public.

Test owner versus client

If the temporary test owner claims the device for commissioning, record that decision and schedule a controlled transfer before client handover. Changing the owner account later requires more than signing out and signing into another account. The administrator procedure is on page 8.

Source: ClaimDeviceScreen.tsx; RegisterScreen.tsx; rtdbService; device-provisioning-guide.md.

Administrator: verify or provision a device

Open the Firebase console and select **hyfepoul-dev**. These steps require the authorized project administrator. Before edits, check the existing records. For this prototype, the provisioning work is already recorded as complete; verify instead of overwriting.

1. Dedicated Auth identity

In Authentication / Users, locate the dedicated ESP32 account configured in the ignored local Config.h. If provisioning a new unit, create its dedicated email/password account through the approved process. Copy its Auth UID privately and confirm the firmware uses that account. Do not use the operator's owner account or repurpose a shared test-device identity.

2. Device binding

Under Realtime Database / Data, inspect **devices/device-001**. A new, unclaimed device needs the following fields. Preserve existing fields for an already provisioned device; do not replace the whole record or root.

Field under devices/device-001	Type	Value / meaning
deviceAuthUid	String	The dedicated device account's Auth UID.
status	String	offline for a newly provisioned unit.
lastHeartbeat	Number	0 before the device has reported.
ownerId	Absent initially	The claim process writes the owner's UID. Do not prefill for an ordinary first claim.

3. Private setup PIN

At **deviceSecrets/device-001/setupPin**, securely set a newly chosen six-digit **string**. In JSON/import syntax, quote the digits to make a string. In a console editor that offers a type selector, select String and enter the digits; do not store quote characters inside the value. Verify the resulting type and preserve leading zeros.

The app cannot read deviceSecrets. It supplies the PIN to the claim request, and database rules compare it with the private string. A numeric value does not satisfy a strict string comparison.

4. Verify the complete relationship

Confirm the firmware Device ID, project and account match the record. Verify an authenticated device binding read and later actual hardware writes. Deliver the Device ID and PIN privately to the intended owner. Do not paste secrets into this manual, logs, screenshots or Git.

Source: docs/device-provisioning-guide.md; backend RTDB rules; Config.example.h.

Technician: connect the actual ESP32

Private configuration and build

Use the ignored local **firmware/esp32/HyFePoul_ESP32/Config.h**. Confirm the Firebase project, Device ID, API key, dedicated account email/password and the real 2.4 GHz Wi-Fi settings. The prepared baseline image still contains Wi-Fi placeholders and must be rebuilt after this edit.

The device password is separate from the setup PIN. A mismatch requires an authorized account-password repair; entering the PIN as the firmware password will not work. Preserve other device accounts and records when repairing this unit.

Keep ESP32 Core Debug Level at None. Build and upload using the Hardware / Technical Manual and the documented board target. Keep the configured ESP32 firmware binary private because it embeds credentials.

Prove the connection with motors disconnected

1. Confirm board power and safe startup. Observe redacted serial output for Wi-Fi association.
2. Confirm network time and TLS succeed, and the dedicated device account signs in.
3. Confirm the sign-in UID matches **devices/device-001/deviceAuthUid**.
4. Confirm heartbeat and device-state writes succeed under the deployed rules.
5. Confirm Mega UART data is valid and the phone displays fresh device readings.
6. Disconnect/reconnect Wi-Fi under the controlled test procedure and record recovery. Do not use a console-written ONLINE value as proof of a working device.

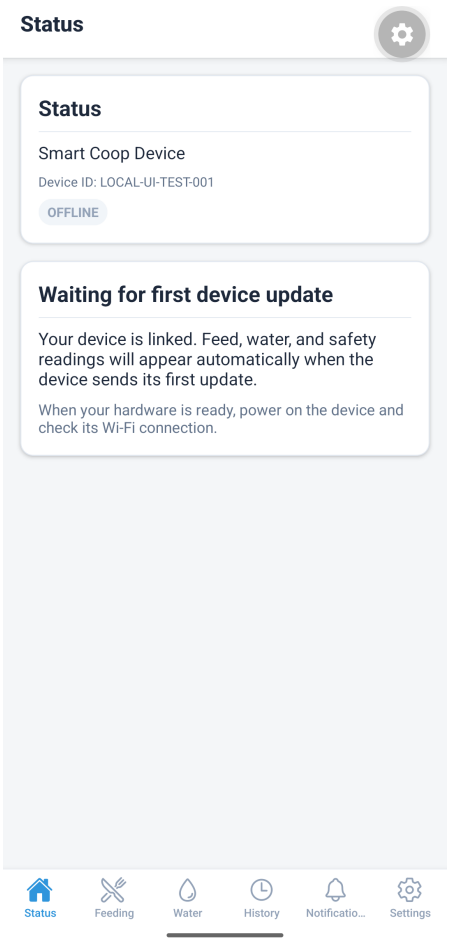
Key and rules baseline

The approved app/device API key for this edition allows Identity Toolkit, Secure Token and Realtime Database. It has no Android application restriction because this implementation uses the Firebase JavaScript SDK and direct ESP32 REST requests. Its value is intentionally omitted. Database authorization still depends on authentication, identity binding and rules.

Do not change production key restrictions or weaken database rules simply to make a test pass. Capture the exact failure and compare the current configuration against the approved build record. Legacy Firestore/Functions/FCM setup is not needed for this edition's deployed path.

Source: development-status.md; firmware-build-environment.md; hardware-test-day-run-sheet.md.

Verify the phone, ownership and live data



Expected before the first hardware update

SCREEN GUIDE

01 A linked device can legitimately be offline. Ownership and hardware connectivity are separate acceptance checks.

02 The first-update message should disappear after the device supplies valid readings. It should not remain a permanent loading spinner.

03 Verify the actual unit's Device ID. This screenshot is a local emulator example and must not be used as a production device identifier.

04 Use actual sensor changes during commissioning to prove the app receives the correct unit's data.

Historical Android emulator capture. Simulated values only; visual details may differ from the installed APK.

Record these checks on the actual phone

[ ] The production APK launches without a development server.

[ ] The intended owner can sign in and claim; the administrator verifies ownerId.

[ ] Fresh heartbeat and valid Mega data reach the correct device record.

[ ] Status, Feeding and Water agree with deliberate, safe sensor changes.

[ ] A schedule edit reaches ON DEVICE and the Mega retains it as expected.

[ ] Command status follows the actual result; no unsafe motion is attempted before the hardware gates pass.

[ ] Notification permission and in-app alert behavior are understood. Closed-app delivery is not promised.

Use HIT-05 and HIT-15 with the detailed commissioning procedure. Later, after safety and calibration checks, verify a full physical feed and the corresponding app history. Do not mark that end-to-end test passed from simulator data or a successful account claim alone.

Source: hardware-test-day-run-sheet.md; mobile-empty-state-qa-2026-10-26.md.

Transfer from test owner to the client

This is an administrator-coordinated handover, not routine troubleshooting. Agree on the new owner, the handover time and retention of test data first. Keep the device out of normal operation while access and schedules are reviewed.

1. Confirm the new client's account exists and identify its UID privately. Confirm the old owner and any unfinished command or schedule changes.
2. Securely set a **fresh six-digit string PIN** for the same device. Share it only with the intended new owner.
3. Remove the old **devices/device-001/ownerId** and the retained **claimRequests/device-001** record through the authorized console procedure. Preserve deviceAuthUid, device configuration and the rest of the database.
4. Have the new client sign in on the phone and claim the device with its exact ID and new PIN.
5. Verify ownerId equals the new client's UID. Confirm the previous owner no longer has device access under the rules.
6. After a successful claim is verified, remove the retained claim request if appropriate to avoid retaining its submitted PIN. Record the handover without secret values.
7. Review all three schedules, device time, pending command state and alert history with the client. Complete the phone acceptance and required physical checks before restoring normal operation.

WHY BOTH RECORDS MATTER

Deleting only ownerId can leave the previous claimRequests entry blocking the next user's claim. Changing the PIN alone also does not transfer an already owned device. Follow the whole sequence and inspect any partial failure before repeating it.

Handover checklist

[ ] Correct owner and device binding verified; test-owner access checked.

[ ] Private PIN delivered through a separate secure channel.

[ ] Configured firmware and production APK identified; Wi-Fi recovery demonstrated.

[ ] Safety, calibration and full feed acceptance recorded in the HIT log.

[ ] Operator understands stop/reset, refill, schedules and notification limits.

[ ] Technician contact, revised manuals and unresolved limitations supplied.

Sources and edition

Edition 1.0, 27 October 2026; baseline d623cce. Main sources: docs/device-provisioning-guide.md, docs/development-status.md, docs/hardware-test-day-run-sheet.md, mobile-app/src/screens/ClaimDeviceScreen.tsx, RegisterScreen.tsx and the RTDB service/rules. Account and hardware status can change after this publication; re-verify the records before edits.